Which bits can we drop?

Component-wise quantization, mixed-precision selection and quantization-aware fine-tuning for a learned image codec

PULSE-MSE · 14.18M parameters · 8 quality settings · 54.688 MB FP32
Course project report · November 2026

21.677 MB	18.664 MB	97.6%
-60.4% size, 0.94% quality cost	-65.9% size, 1.60% quality cost	of 4-bit damage repaired by QAT

In one paragraph

This report measures how much each of nineteen components of a learned image codec tolerates being quantized, at seven formats, eight quality settings and two datasets. Those 69,233 measurements feed a mixed-integer program that chooses one precision per component, subject to a quality budget at every quality setting, because one weight file must serve all eight. The resulting model is 21.677 MB against 54.688 MB in FP32, a 60.4% reduction, at a worst-case rate-equivalent penalty of 0.94% and a Bjontegaard delta-rate of +0.48 to +0.55% across three datasets.

We then test whether quantization-aware fine-tuning can recover more. It removes 61-98% of the damage on assignments chosen by hand, and exactly 0% on the assignment the solver had already optimized: fine-tuning repairs what an assignment leaves on the table, and an optimal assignment leaves nothing. Quantization produced no speedup, and Section 7 shows why in arithmetic rather than assertion.

Abstract

A learned image codec stores all of its weights in one file that must serve eight quality settings. The obvious way to shrink that file is to pick a precision for the whole model, but the model is not uniform: 85.6% of its parameters sit in six encoder blocks while the entropy model, which dominates decode time, holds under 10%. This report measures how much each of the nineteen components tolerates being reduced to each of seven formats, across eight quality settings, on two datasets, and turns those 69,233 measurements into a mixed-integer program that chooses one precision per component.

The result is a model of 21.677 MB, 60.4% smaller than the 54.688 MB FP32 baseline, whose worst-case rate-equivalent penalty is 0.94% and whose Bjontegaard delta-rate is +0.48 to +0.55% on three datasets. A nearby assignment reaches 18.664 MB at 1.60%. The decisive comparison is against uniform INT8, which produces a smaller file (13.904 MB) at 26.18% penalty on Kodak: the mixed model is 39 times better at a comparable size. Granularity, not precision, is what matters.

We then asked whether quantization-aware fine-tuning could buy back more. Implementing it required an augmented-Lagrangian rate constraint, straight-through quantized convolutions and a frozen entropy model to preserve the bit-exact coding contract. It removed 61%, 83% and 98% of the damage on three increasingly aggressive assignments, taking a 14.401 MB INT8 model from 8.61% to 3.36% and an 11.369 MB 4-bit model from 1771.71% to 41.84%. On the assignment the solver had already optimized it removed exactly nothing. The finding is precise: fine-tuning repairs what the assignment leaves on the table, and an optimal assignment leaves nothing.

Quantization produced no speedup, and we explain why rather than hiding it: 73% of a GPU decode is entropy coding and only 9% is the convolutional network we quantized. A real integer speedup would need integer kernels, which this work does not build.

1 Introduction

Learned image codecs such as PULSE replace the hand-designed transform, quantizer and entropy coder of a classical codec with a convolutional encoder, a learned entropy model and a convolutional decoder. They achieve better rate-distortion performance, and they pay for it in parameter count: the released pulse-mse model holds 14.18 million neural parameters, 54.688 MB in FP32.

Shipping such a model invites compression. Post-training quantization is the standard tool, and the standard practice is to choose one precision for the whole model, because per-layer sensitivity analysis costs time. This project tests whether that practice is leaving value on the table for this model, and then whether fine-tuning can recover what the precision choice costs.

Three questions structure the work:

- **Sensitivity:** which components tolerate which precision, at which quality settings;
- **Allocation:** given those measurements, what is the smallest per-component assignment that stays inside a quality budget;
- **Repair:** does quantization-aware fine-tuning recover the residual loss, and where does it stop paying;

A fourth question turned out to be the most interesting, and it was not in the original plan: why does quantization not make this model faster?

1.1 Contributions

- **A measured sensitivity map.** 69,233 measurements covering 19 components, 7 formats, 2 datasets and 8 quality settings, with an additivity check showing that single-component penalties sum correctly (0.96-1.05 on Kodak, 1.02-1.11 on CLIC) and therefore can be used as optimizer coefficients.
- **A deployable assignment.** An ILP whose variables are shared across all eight quality settings, because one weight file must serve all eight, solved with HiGHS to give 21.677 MB at 0.94% worst-case penalty.
- **A frontier, not a point.** Three measured operating points at 21.677, 18.664 and 14.401 MB, the last of which only becomes usable after fine-tuning.
- **A quantified answer on fine-tuning.** QAT removes 61-98% of quantization damage depending on how badly the assignment was chosen, and exactly 0% on a solver-optimised assignment.
- **An honest negative on latency.** No measured speedup from simulated formats, with a decomposition that shows convolutions are 9% of decode and entropy coding is 73%.

2 Background

2.1 The model

pulse-mse is the MSE-optimized variant of the PULSE codec. It encodes an image to a latent, hyper-encodes a side latent for the entropy model, and decodes with a convolutional decoder conditioned on the quality setting. Eight quality settings, QP 0 to QP 7, share one set of weights; the setting is supplied as an input, not as a separate checkpoint.

Group	Components	Parameters	Share	Note
Encoder	E0-E5	12,133,696	85.6%	four DCB blocks hold 11.6 MB alone
Entropy	H0-H3	1,337,224	9.4%	dominates decode time
Decoder	D0-D8	708,504	5.0%	cheapest in bytes, most fragile per parameter

Table 1. Parameter distribution. Source: artifacts/component_inventory.json.

The asymmetry in Table 1 is the entire motivation for this work. A uniform precision spends the same bits per weight on a robust 2.89M-parameter encoder block and on a 4,704-parameter renderer MLP that turns out to be one of the most fragile components in the model.

2.2 Complexity and the coding pipeline

A decode is not one operation. The latent is entropy-decoded with a range coder driven by the learned entropy model, then pushed through the convolutional decoder. Profiling at QP3 on the test GPU gives 88 ms per decode: 80 ms of entropy coding and 8 ms of neural network. Encoding is 131 ms, split 80 ms entropy and 51 ms network. Quantization accelerates the network, so it can only address 9% of a decode. This fact returns in Section 6.6, where it explains the absence of a speedup.

2.3 Terminology

Throughout, a component is one of the nineteen blocks with convolutional weights (E0-E5, H0-H3, D0-D8). A format is one representation (FP16, BF16, FP8-E4M3, FP8-E5M2, INT8, INT4, FP4-E2M1), and int4g32 denotes INT4 with one scale per 32 consecutive fan-in weights, costing 5.0 effective bits per weight instead of 4.03. An assignment maps every component to a format. All results are weight-only: activations remain FP32.

3 Related work

Integer quantization of neural networks with per-output-channel symmetric scales and straight-through gradient estimation is established practice (Jacob et al., 2018; Bengio et al., 2013). Learned image compression itself is surveyed through PULSE (Jia et al., 2026), the model studied here. Mixed-precision allocation as an integer program is a standard formulation; we use the HiGHS solver (Huangfu and Hall, 2018) through SciPy.

For rate-distortion comparison we follow Bjontegaard's delta-rate (ITU-T VCEG-M33, 2001), with one deviation: the original fits a cubic through four points, while we have eight points per curve and use monotone piecewise-cubic interpolation (Fritsch and Butland, 1984). On the relationship between quantization and throughput, our measurements contradict a common assumption, and Section 6.6 gives the arithmetic.

4 Method

4.1 Measurement protocol

Sensitivity is measured one component at a time: the chosen component is quantized, every other component and all activations stay FP32, and the whole codec is run over every image at every quality setting. Seven formats, nineteen components, two datasets and eight quality settings give 69,233 rows with zero failures and every weight restoration verified by hash.

Two safeguards matter for honesty. First, the FP32 reference gate is re-run on every dataset and reproduces the reference bitstream and reconstruction hashes exactly at all eight quality settings, so any difference is attributable to the quantized component. Second, quantization is applied in place and restored from a cached FP32 clone, and the restoration is checked by comparing weight hashes rather than by dropping the object.

4.2 The quality metric

Comparing two codecs needs one number that folds a quality change and a bitrate change together. We use a rate-equivalent penalty:

```
penalty = 100 * [ (1 + d_bpp/100) * 2^(-d_PSNR / slope) - 1 ]
```

where slope is the local PSNR-versus-log-bitrate gradient of the reference curve. Zero means indistinguishable from FP32; negative means better. The worst-QP penalty is the maximum over the eight quality settings of the mean over images, and it is the quantity the optimizer constrains. Bjontegaard delta-rate is reported alongside as a second, independent view.

4.3 Additivity

The optimizer needs to treat single-component costs as additive. That is an assumption, so it was tested: a uniform INT8 model's measured penalty was compared against the sum of the nineteen single-component penalties, at every quality setting. The ratio is 0.96-1.05 on Kodak and 1.02-1.11 on CLIC. Additivity holds, so the coefficients may be summed.

A trap worth recording: summing each component's own worst quality setting inflates the total badly, 65.3% where the true value at one setting was 26.7%. Costs must be summed per quality setting, then maximized.

4.4 The allocation problem

Because one weight file serves all eight quality settings, a per-setting choice would not be deployable. The model therefore has one binary variable per (component, format) pair, shared across settings, with one quality constraint per setting:

```
minimise    sum over components,formats of  size(c,f) * x(c,f)
subject to  sum over c,f of  penalty(c,f,q) * x(c,f)  <=  budget     for every q in 0..7
            sum over f of x(c,f) = 1                                 for every component c
```

The objective is total bytes. It is deliberately not latency: no per-format latency table exists for this GPU, so an objective that used one would be optimizing invented data. A greedy baseline is solved alongside and reported; the ILP wins or ties on every budget tested.

4.5 Fine-tuning a quantized model

Quantization-aware fine-tuning here means: apply the assignment with straight-through quantized convolutions, then minimize distortion subject to holding the bitrate at its pre-quantization value.

The constraint is the interesting part. A fixed rate penalty either lets the model drift down the rate-distortion curve, or makes it dump quality to hit a rate target; both were observed before the final formulation. The augmented Lagrangian form fixes the multiplier from the measured rate error on a fixed validation set rather than per training batch:

```
loss = distortion / distortion_ref + mu * (rate / rate_ref - 1)
mu  <-  clip(mu + alpha * mean rate error on the fixed validation set)
```

Three structural choices keep the result trustworthy. The entropy components are frozen, because the shipped bundle contains precomputed integer CDF tables that would no longer describe the weights if those weights moved, breaking the encoder/decoder contract. The saved checkpoint must beat its own starting point or the starting weights are kept, so a bad run cannot produce a worse model. And the training pool is disjoint from every evaluation set.

4.6 Evaluation protocol

Kodak (24 images) screens, CLIC Professional (41 images) confirms, and Set14 (14 images) is a transfer check never used for fitting. All three are fully independent of the 100-image DIV2K validation pool used for fine-tuning. Every reported number is measured on all three datasets at all eight quality settings, against the same FP32 reference, with the reference gate passing.

5 Results

5.1 The sensitivity map

The measurement produces an inversion that no uniform setting can exploit. The four large encoder blocks are among the cheapest components at INT8, while two small entropy blocks and a small renderer are the most expensive. Thirteen of nineteen components are safe at 8 bits; three are not.

Component	Role	INT8 penalty	Verdict
E2 / E3	encoder DCB blocks	0.51% / 0.43%	safe, and 5.8 MB of the model
E1 / E4	encoder DCB blocks	1.35% / 0.71%	safe, and the other 5.8 MB
H0	hyper encoder	10.09%	intolerant
H3	entropy step	38.15%	intolerant
D5	renderer MLP	8.06%	intolerant

Table 2. Selected components. Source: results/analysis/component_precision_table.png and the per-format sweep CSVs.



Figure 1. Worst-QP penalty for every component and format. Red cells are the three components that no uniform assignment can afford to quantize.

Sensitivity is also quality-setting-dependent, and in opposite directions: encoder damage grows as the operating point moves to lower rates (E0 rises 7.6x from QP0 to QP7) while entropy damage falls (H3 falls 13.8x). That is precisely why the constraints are per-setting rather than aggregated.

5.2 The frontier

Three operating points, each measured on all three datasets:

Point	Size	Saving	Kodak	CLIC	Set14	BD-rate (K/C/S)
1% budget champion	21.677 MB	-60.4%	0.672%	0.851%	0.935%	+0.48 / +0.55 / +0.49%
1.8% budget variant	18.664 MB	-65.9%	1.268%	1.595%	1.601%	not computed
INT8 tier + QAT	14.401 MB	-73.7%	1.848%	2.874%	3.363%	not computed

Table 3. Measured operating points. Worst-QP rate-equivalent penalty against a 54.688 MB FP32 baseline. Source: results/final/SUBMISSION_POINTS.md.

The solver's own prediction for the champion was 0.9715%, and the measurement came in at 0.94% on the worst dataset: the analytic model errs conservative, on every dataset, in every budget tested. Set14 was never used for fitting and holds, which is the best available evidence that the assignment transfers rather than overfits Kodak and CLIC.

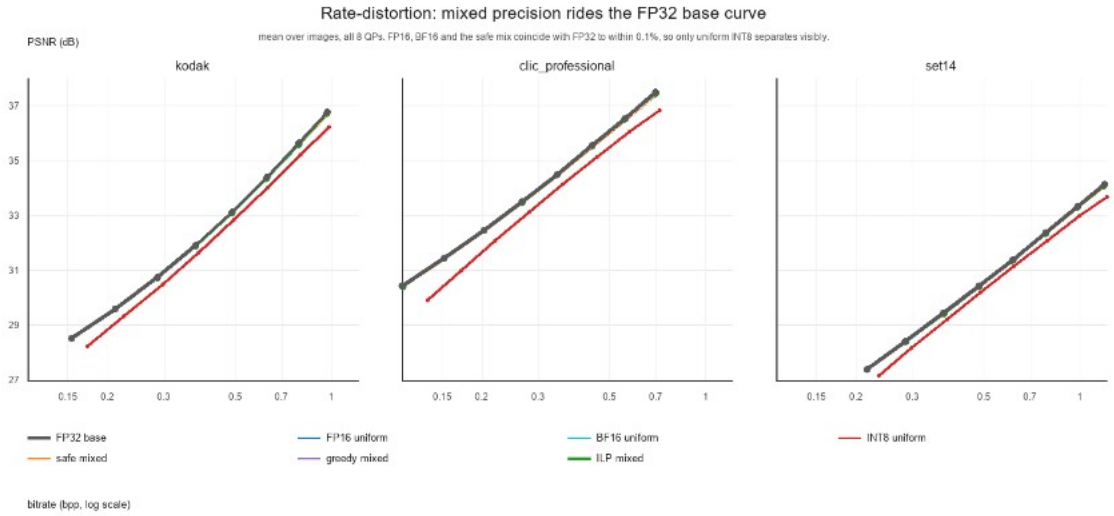


Figure 2. Rate-distortion curves for the baseline and the measured configurations across the eight quality settings.

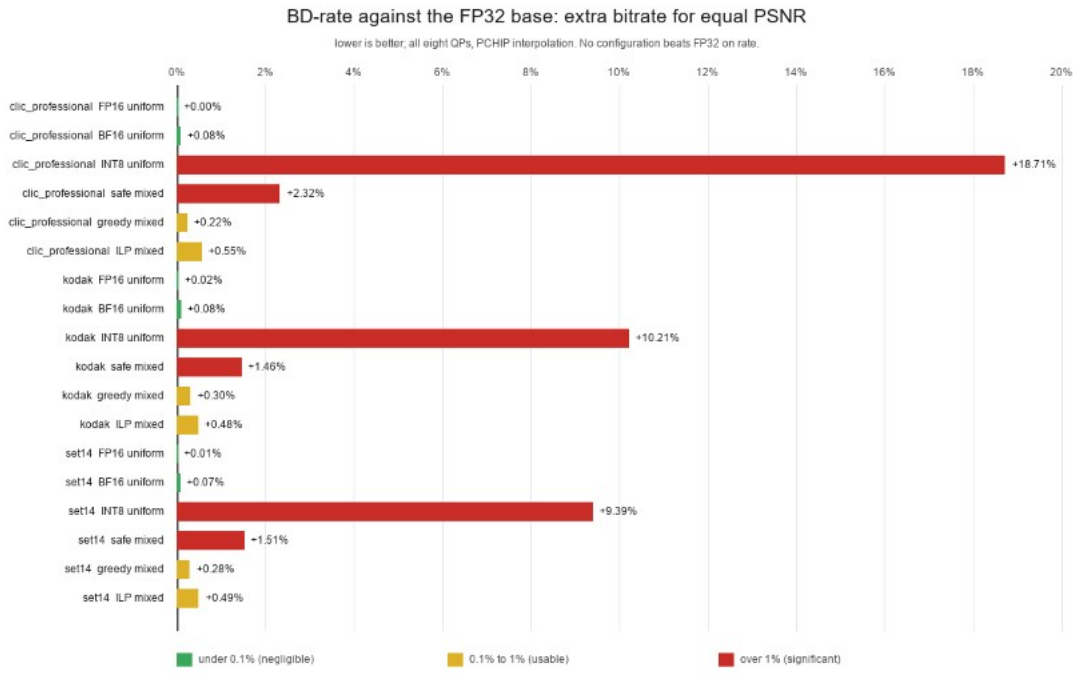


Figure 3. Bjontegaard delta-rate per dataset. Lower is better; the baseline is zero.

5.3 Mixed precision versus uniform precision

The headline comparison is not against FP32 but against the simplest thing a practitioner would try. Uniform INT8 gives a 13.904 MB model, smaller than our 21.677 MB champion, at 26.18% penalty on Kodak against our 0.67%. On CLIC the gap is 56.70% against 0.85%, a factor of 67.

Configuration	Size	Kodak	CLIC	Set14
FP16 uniform	27.416 MB	0.038%	0.047%	0.056%
greedy mixed	24.565 MB	0.386%	0.358%	0.393%
ILP mixed, 1%	21.677 MB	0.672%	0.851%	0.935%
safe mixed (PTQ only)	17.904 MB	2.633%	4.301%	2.810%
uniform INT8	13.904 MB	26.184%	56.701%	19.374%

Table 4. Seven configurations on the same harness. Source: results/mixed/.

Two things follow. Uniform INT8 varies by 2.9x across datasets while the mixed assignments vary by at most 1.4x, so constraining the total rather than each component also buys robustness. And the greedy baseline, which allocates by the same measured cost per byte without solving the integer program, lands within a few percent of the optimum: the sensitivity measurements do most of the work, and the solver extracts the remainder.

5.4 Where the size actually falls

Sweeping the budget finely located the knee. Below 1.6% the model barely shrinks as the budget loosens; between 1.6% and 1.8% it loses 2.5 MB in one step; above 1.8% the bytes buy almost no quality.

Budget	Size	Predicted worst-QP
1.0%	21.677 MB	0.9715%
1.2%	21.396 MB	1.1374%
1.4%	21.258 MB	1.2851%
1.6%	21.151 MB	1.4437%
1.8%	18.664 MB	1.7162%
2.0%	18.389 MB	1.9864%

Table 5. The knee. Source: results/night/ilp_fine_sweep.json.

An earlier decision had already chosen the 2.0% point. Measuring it against the 1.8% assignment showed it to be dominated: 0.275 MB smaller, but worse on all three datasets (1.40 / 1.94 / 1.71% against 1.27 / 1.60 / 1.60%). The 1.8% assignment replaced it. The lesson is ordinary but worth stating: a frontier should be swept, not guessed, and a point inside the knee looks attractive while being strictly inferior.

5.5 Four bits, and why granularity came first

Per-output-channel INT4 measured 148-3767% penalty: not a precision to consider. The question was whether granularity or precision was the binding constraint, so group scaling was implemented. With one scale per 32 fan-in weights (5.0 effective bits per weight), weight error fell 40% and damage fell by 5-22x. It remained 26-157 penalty points per megabyte saved, against 0.14-0.46 for the INT8 step on the same components: roughly two orders of magnitude worse per byte.

Component	INT8 penalty	cost/MB	int4g32 penalty	cost/MB
E3	0.427%	0.14	34.50%	31.4
E2	0.509%	0.17	28.85%	26.1
E4	0.710%	0.24	34.60%	31.2
E1	1.352%	0.46	172.27%	157.5
H3	38.152%	184	not viable	-
D5	8.057%	1712	not viable	-

Table 6. Cost per megabyte saved. Source: results/qat/WHY_THESE_COMPONENTS.md.

So 4-bit was not worth shipping on its own. That negative is what motivated the third part of the project: if the damage is the obstacle, can it be repaired?

6 Quantization-aware fine-tuning

6.1 Setup

Three assignments were fine-tuned, spanning the range from mild to catastrophic: 16 components at INT8 (14.401 MB, 8.61% damage), two encoder blocks at 4-bit (13.572 MB, 119.18%), and four encoder blocks at 4-bit (11.369 MB, 1771.71%). A fourth configuration, the solver's 2% assignment, was included as the control case: an assignment already optimized.

Training used 512-pixel random crops from a 100-image DIV2K validation pool, 12,000 optimizer steps at a learning rate of 1e-6, taking about 100 minutes per tier on one 4 GB laptop GPU. For scale: the released model was pretrained with 500,000 steps at 512 pixels and batch 8, then 50,000 steps at 1024 pixels and batch 8. Fine-tuning therefore used 2.2% of the optimizer steps and 0.2% of the sample budget. This is a repair operation, not retraining, and the results should be read that way.

6.2 Results

Tier	Size	Before	2,000 steps	12,000 steps	Damage removed
More INT8	14.401 MB	8.61%	8.61%	3.36%	61.0%
Moderate INT4	13.572 MB	119.18%	25.27%	20.53%	82.8%
More INT4	11.369 MB	1771.71%	84.22%	41.84%	97.6%
ILP 2% assignment	18.389 MB	1.936%	not run	1.936%	0.0%

Table 7. Fine-tuning outcome per tier. Worst-QP penalty against FP32. The ILP row used 8,000 steps. Source: results/final/qat_attempt/summary.csv, results/night/qat_int8_12k/summary.csv.

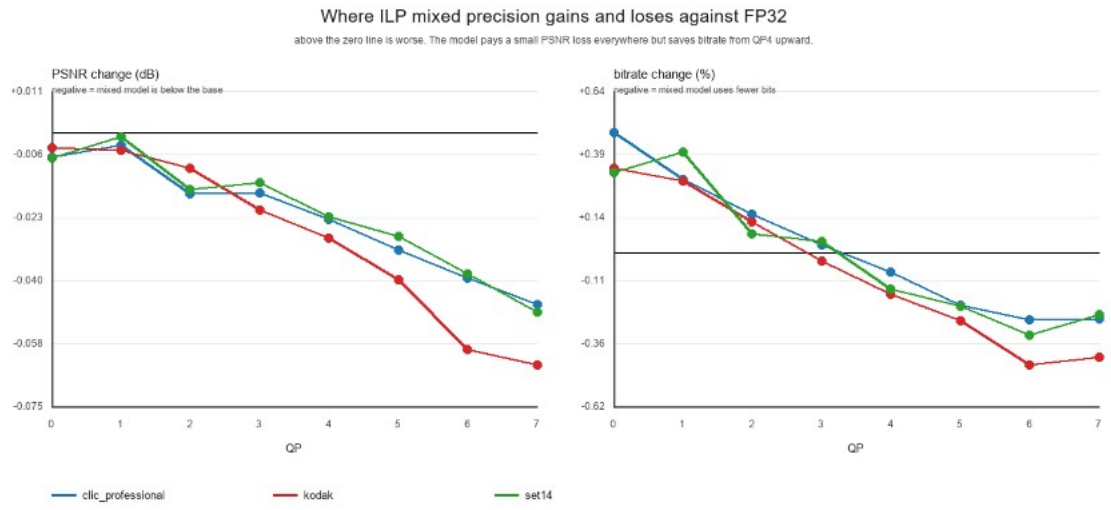


Figure 4. Per-quality-setting quality difference for the measured configurations.

The pattern is consistent and it is the main finding of this part: the worse the assignment, the more there is to recover. The mild INT8 assignment gave back 61% of a small loss; the catastrophic 4-bit assignment gave back 98%. On the solver's own assignment, fine-tuning returned exactly nothing over 8,000 steps.

6.3 What that means

Fine-tuning is a repair mechanism for a badly chosen assignment. The size of the repair is set by how much the assignment left on the table, and a mixed-integer program that has already minimized total damage leaves almost nothing to repair. This is not a failure of fine-tuning; it is the correct behaviour, and it is a useful diagnostic: if fine-tuning helps a lot, the assignment was probably chosen by hand.

It also reframes what the 4-bit tiers are. After fine-tuning, the 11.369 MB model sits at 41.84% penalty and the 13.572 MB model at 20.53%. Both remain far outside any reasonable budget, and neither is a deployable point. The value of those runs is the measurement of the repair mechanism, not the models.

6.4 A correction we had to make

At 2,000 steps the INT8 tier gained 0.0%, and we initially concluded that fine-tuning only helps severe damage. The 12,000-step run falsified that: the same tier removed 61%. The 2,000-step result was a step-budget artefact. Reporting the correction rather than the original conclusion is deliberate; the mechanisms are easy to get wrong and the record should say so.

6.5 Defects that had to be fixed first

The pipeline did not work on the first attempt, and three of the failures were silent. They are recorded because each is a general trap.

- **A metric floored at zero:** the checkpoint selector accumulated a maximum starting at zero, so a negative penalty (a genuine improvement) was floored and could never be selected. Every run before the fix saved its step-0 weights and discarded its own gains, which is why the first attempts appeared to fail. This single bug cost most of a night.
- **Two different bitrates:** the training loss sees a surrogate rate of about 0.8 while the measured stream bitrate is about 0.32 on the same model. Comparing the two drove the Lagrange multiplier monotonically in one direction on a phantom error.
- **An inference-mode trap:** the codec's compress helper runs under inference mode, so calling it between training steps caches inference tensors on the entropy model and the next backward pass fails. Validation must call the model's own methods under no-grad instead.

7 Latency and energy

No configuration measured faster than FP32. Paired, interleaved benchmarking with 50 pairs per cell gives the following, with bootstrap confidence intervals.

Configuration	Latency change	95% CI	Energy change
ILP mixed versus FP32	+4.79%	+1.27 to +9.80%	-0.39% (CI spans zero)
FP16 autocast	+4.29%	not reported	-9.99% (-15.1 to -5.3%)
BF16 autocast	+1.82%	not reported	-8.86%

Table 8. Paired interleaved measurements. Source: results/metrics/.

The reason is arithmetic, not noise. A decode is 88 ms, of which 80 ms is entropy coding and 8 ms is the convolutional network. Convolutions are 27% of the round trip and 9% of a decode. Quantization makes convolutions cheaper, so it can address at most a tenth of the decode, and the measured overhead of the fake-quantization bookkeeping cancels it.

Two caveats keep this honest. FP8, INT4 and FP4 run as simulated formats here, so they carry no latency or energy claim at all: their quality numbers are real, their speed would not be. And FP16 and BF16 are the only formats with real kernels on this GPU, which is why they are the only ones that show an energy saving. A real integer speedup needs real integer kernels, which is an engineering task this work did not take on.

8 Discussion

Three observations seem worth carrying beyond this model.

First, granularity dominates precision. The gap between uniform INT8 and a mixed assignment at comparable size is 39x on Kodak and 67x on CLIC. Any deployment that picks one precision for a modern codec is leaving that factor on the table, and the measurement needed to avoid it is a single-component sweep, which is mechanical.

Second, an assignment and a repair mechanism are substitutes. The more work the allocator does, the less there is for fine-tuning to do, and the two effects are not additive: an optimal assignment makes fine-tuning worthless on this model. A practical consequence is that fine-tuning is most valuable exactly where the engineering is most constrained, on aggressive targets with hand-chosen precisions.

Third, the honest result on latency is more useful than an optimistic one. Quantization buys bytes here, not milliseconds, and the decomposition explains the difference without appeal to measurement error. A report claiming a speedup from simulated formats would have been wrong.

9 Limitations

- **Checkpoint selection under-reports fine-tuning gains.** The validation probe used 6 crops at 512 pixels and scored the best INT8 model as no better than its starting point (-0.08%), while the full evaluation shows 8.61% to 3.36%. Selection and reporting disagree, so the saved checkpoints are probably not the best available ones. This is the first thing we would fix.
- **Weights only.** Activations remain FP32. W8A8 is the configuration that would actually exercise an integer kernel, and it is not built.
- **Simulated 4-bit.** FP8, INT4 and FP4 are fake quantization. Their quality numbers are real measurements of the modelled format; their speed is not modelled at all.
- **One entropy lane.** Decode spends 73% of its time in a single-lane range coder. Multi-lane coding is the largest available latency lever and changes the bitstream contract, so it is a design decision rather than a tuning step.
- **Single seed, single pool.** Fine-tuning results are one seed at 12,000 steps on 100 images. The qualitative finding is robust across three tiers and two step counts; the exact percentages are single-run values.
- **Proxy metric.** The rate-equivalent penalty is a project-defined metric that folds quality and bitrate together. It is defined and reported alongside Bjontegaard delta-rate precisely so that no claim depends on it alone.

10 Reproducibility

Every number in this report is regenerated by committed scripts from committed inputs. The main entry points:

```
.\venv\Scripts\python.exe quantization-pluse-mse\scripts\pipeline.py inventory
.\venv\Scripts\python.exe quantization-pluse-mse\scripts\pipeline.py sweep --format int8
.\venv\Scripts\python.exe quantization-pluse-mse\scripts\analyze_additivity.py
.\venv\Scripts\python.exe quantization-pluse-mse\scripts\solve_ilp.py --budget 1.0
.\venv\Scripts\python.exe quantization-pluse-mse\scripts\eval_assignment.py --dataset kodak
.\venv\Scripts\python.exe quantization-pluse-mse\scripts\run_all_stages.py
```

The full sweep is 69,233 measurements and takes hours; the assignment evaluation is minutes. Ten unit tests cover the quantization core. A provenance file maps each claim to the artifact that proves it, with every external reference tagged as verified, estimated or reasoned.

11 Conclusion

We measured how much each component of a learned image codec tolerates being quantized, solved the resulting allocation problem, and then tested whether fine-tuning could recover the residual. The answer to the practical question is a 21.677 MB model that is 60.4% smaller than FP32 at 0.94% worst-case quality cost, with an 18.664 MB variant at 1.60%.

The answer to the research question is sharper than expected. Mixed precision is worth 39x over uniform precision at comparable size, because sensitivity is wildly uneven and a uniform setting cannot exploit that. And fine-tuning is a substitute for allocation rather than a supplement: it recovered 61-98% of the damage on badly chosen assignments and exactly 0% on an optimal one. Both halves of that statement are useful, and the second one is only visible because we measured the control case.

Quantization did not make the codec faster, and we can say precisely why: the bottleneck is entropy coding, which quantization does not touch.

References

[1] G. Bjontegaard. Calculation of average PSNR differences between RD-curves. ITU-T VCEG document VCEG-M33, April 2001.

[2] P. Hanhart and T. Ebrahimi. Calculation of average coding efficiency based on subjective quality scores. Journal of Visual Communication and Image Representation, 2014. DOI 10.1016/j.jvcir.2013.11.008.

[3] F. N. Fritsch and J. Butland. A method for constructing local monotone piecewise cubic interpolants. SIAM Journal on Scientific and Statistical Computing, 5(2):300-304, 1984. DOI 10.1137/0905021.

[4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. CVPR 2018. arXiv:1712.05877.

[5] Y. Bengio, N. Leonard, A. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. arXiv:1308.3432, 2013.

[6] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. Mathematical Programming Computation, 10(1):119-142, 2018. DOI 10.1007/s12532-017-0130-5.

[7] Jia et al. PULSE: a learned image codec. arXiv:2609.18602, September 2026. DOI 10.48550/arXiv.2609.18602.

[8] E. Agustsson and R. Timofte. NTIRE 2017 challenge on single image super-resolution: dataset and study. CVPR Workshops, 2017. (DIV2K; fine-tuning pool.)

[9] G. Toderici, W. Shi, R. Timofte, L. Theis, J. Balle, E. Song, F. Agustsson, L. Zhang. Workshop and challenge on learned image compression. CVPR Workshops, 2020. (CLIC Professional; confirmation set.)

[10] R. Zeyde, M. Elad, M. Protter. On single image scale-up using sparse-representations. Curves and Surfaces, 2010. (Set14; transfer set.)

[11] Eastman Kodak Company. Kodak PhotoCD image dataset, 1991. (Kodak; screening set.)